## 3.2 App Development

The following debugging examples refer to the Flutter library, which allows you to create apps for iOS and Android as well as web applications.

### 3.2.1 Flutter and Null Safety Values

The mobile app we created using the Flutter library was supposed to be multilingual. As a modern toolkit, Flutter naturally provides the tools for this, but we did run into a small problem: the application would no longer compile because the translation entries aren't *null safe*. Roughly speaking, the compiler can't ensure at compile time that the translation object is available at runtime.

The detailed error messages of the dart compiler already give a clear indication of what to do, but we also wanted to let the AI tool solve the problem:

**Prompt:** Flutter: My app isn't compiling after I inserted translations for labels.

```
ERROR: lib/widgets/team.dart:43:62: Error: Property 'warning'
  cannot be accessed on 'AppLocalizations?' because it is
  potentially null.
  - 'AppLocalizations' is from
  'package:flutter_gen/gen_l10n/app_localizations.dart'
  ('.dart_tool/flutter_gen/gen_l10n/app_localizations.dart').
  Try accessing using ?. instead.
  title: Text(AppLocalizations.of(context).warning),
                                           ^^^^^^^
```

> This error occurs because the `AppLocalizations.of(context)` call might return null, and Dart's null safety feature is preventing you from accessing the warning property on a potentially null object. To fix this, you can use the null-aware operator `?.` as suggested in the error message. Here's how you can modify your code:
>
> ```
> title: Text(AppLocalizations.of(context)?.warning ?? 'Warning')
> ```

As can be seen from the error message, the *null-aware operator*, that is, `?.`, should be used for the `AppLocalizations`. However, the AI tool goes one step further and appends the replacement text `'Warning'` (the double question mark provides the fallback for the `null` value in Flutter/Dart). Without this change, the app would still not compile because the surrounding `Text()` function expects a string, not `null`.

The AI tool also briefly explained the background to the changes, which we found very helpful:

> This change does two things:
>
> 1. It uses the `?.` operator to safely access the warning property only if `AppLocalizations.of(context)` is not `null`.
> 2. It provides a fallback string (`'Warning'`, in this case) using the `??` operator in case `AppLocalizations.of(context)` is `null` or `warning` isn't available.

Once again, we were pleasantly surprised by the AI tool's debugging capabilities.

### 3.2.2 Layouts in Flutter

App development using the Flutter library requires some rethinking in web development. If areas aren't completely filled by widgets or if they exceed the available space, Flutter fills the area with a black-and-yellow striped background to indicate the problem. This helps us to recognize the problem, but not to solve it, unfortunately. Let's see if AI can help us out here too:

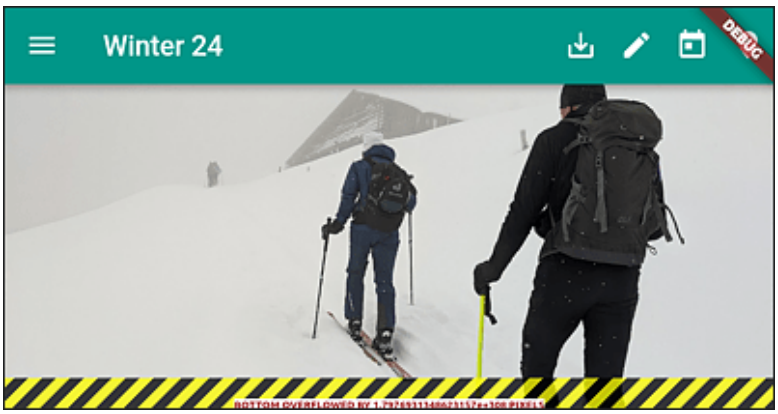


**Figure 3.4** The Flutter Display for a Faulty Layout at the Bottom Edge

**Prompt:** Flutter: I get a yellow-black striped area at the bottom of my app. The widget I am using looks like this:

```
  return Scaffold(
    drawer: const MyDrawer(),
    body: Column(
      children: <Widget>[
        SizedBox(
          width: MediaQuery.of(context).size.width,
          height: 250,
          child:
              Image.file(File(state.currentEntry.image),
                fit: BoxFit.cover),
        ),
        const TagsWidget(),
        MarkdownPreview(
          entry: state.currentEntry,
          picturePath: state.picturePath,
        ),
      ],
    ),
    appBar: AppBar(
        title: Text(state.currentTrip.title),
        actions: action.getActions(context)),
  );
```

> The yellow-black striped area at the bottom of your app typically indicates an overflow error in Flutter. This happens when a widget tries to render content that doesn't fit within its allocated space. In your case, the issue is likely caused by the Column widget in the Scaffold's body. [more details]

Claude 3.5 Sonnet explains two options in the following, both of which lead to the desired result. The generated code can be replaced without changes and it works.

The very brief explanation of the two different solutions (one with a `ListView` and one with a `SingleChildScrollView`) leaves us in the dark as to which is the better solution for our application. So, we ask the following:

**Prompt:** What is the difference between a `ListView` and a `SingleChildScrollView`?

> `ListView` and `SingleChildScrollView` are both scrollable widgets in Flutter, but they have some key differences:

> 1. Child widgets:
>    1. `ListView` is designed to efficiently handle a potentially large number of children. It can have multiple child widgets.
>    2. `SingleChildScrollView` can only have a single child widget, typically a `Column` or `Row`.
> 2. Lazy loading:
>    1. `ListView` supports lazy loading. It only builds the child widgets that are currently visible on the screen, which is more memory-efficient for long lists.
>    2. `SingleChildScrollView` builds all its content at once, which can be less efficient for large amounts of content.
>
> [more details]

Then follows additional points that succinctly describe the differences. Thus, AI doesn't disappoint us here either: even upon reading the first points, we realize that the second variant with the `SingleChildScrollView` is more suitable for our element, which is too large.